associated with self in their current context. In the context of the instance variables defined in the class body, self is the class, and therefore instance variables defined in the class body outside of methods are associated with the class object, not with instances of the class.

Another Way to Access the Singleton Class

We've seen how you can create methods in an object's singleton class by adding the object reference to the method definition using something like def animal.speak.

You can also access the singleton class using Ruby's class << an_object notation:

```
animal = "dog"
class << animal
  def speak
    puts "The #{self} says WOOF!"
  end
end

animal.speak
```

produces:

```
The dog says WOOF!
```

Inside this kind of class definition, self is set to the singleton class for the given object (animal in this case). Because class definitions return the value of the last statement executed in the class body, we can use this fact to get the singleton class object:

```
animal = "dog"
def animal.speak
  puts "The #{self} says WOOF!"
end

singleton = class << animal
  def lie
    puts "The #{self} lies down"
  end
  self     # << return singleton class object
end

animal.speak
animal.lie
puts "Singleton class object is #{singleton}"
puts "It defines methods #{singleton.instance_methods - animal.methods}"
puts "You can also access it as #{animal.singleton_class}"
puts "And the list of methods as #{animal.singleton_methods}"
```

produces:

```
The dog says WOOF!
The dog lies down
Singleton class object is #<Class:#<String:0x00000001024085a0>>
It defines methods []
You can also access it as #<Class:#<String:0x00000001024085a0>>
And the list of methods as [:speak, :lie]
```

Note the notation that Ruby uses to denote a singleton class: #<Class:#<String:...>>.

Ruby goes to some trouble to stop you from using singleton classes outside the context of their original object. For example, you can't create a new instance of a singleton class: